Bridge Pattern

The Bridge Pattern is a structural design pattern that aids in the separation of an abstraction (high-level control logic) from its implementation (low-level functional logic), allowing both to be varied independently. This separation increases modularity and improves code maintainability.

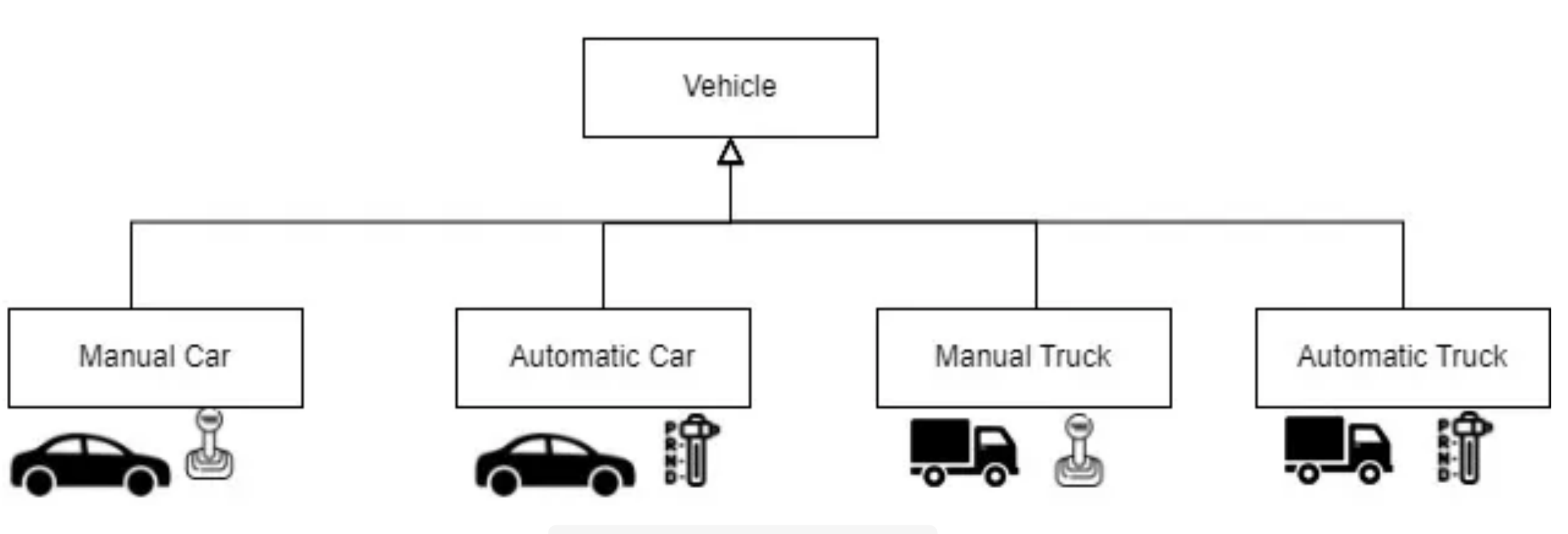
We can better understand this pattern by using an example problem and its solution.

Consider an example of a class hierarchy involving some vehicles and their transmission systems. We will go through this class hierarchy with and without the use of bridge pattern.

Scenario without bridge pattern

- Base Class: Vehicle
- Sub Classes: Car, Truck

Next, if you want to incorporate different transmission systems for vehicles like `Automatic` and `Manual`, you will be required to create different combinations like `automaticCar` , `manualCar`, `automaticTruck`, and `manualTruck`. This is illustrated in the figure below:



Bridge Pattern - Problem

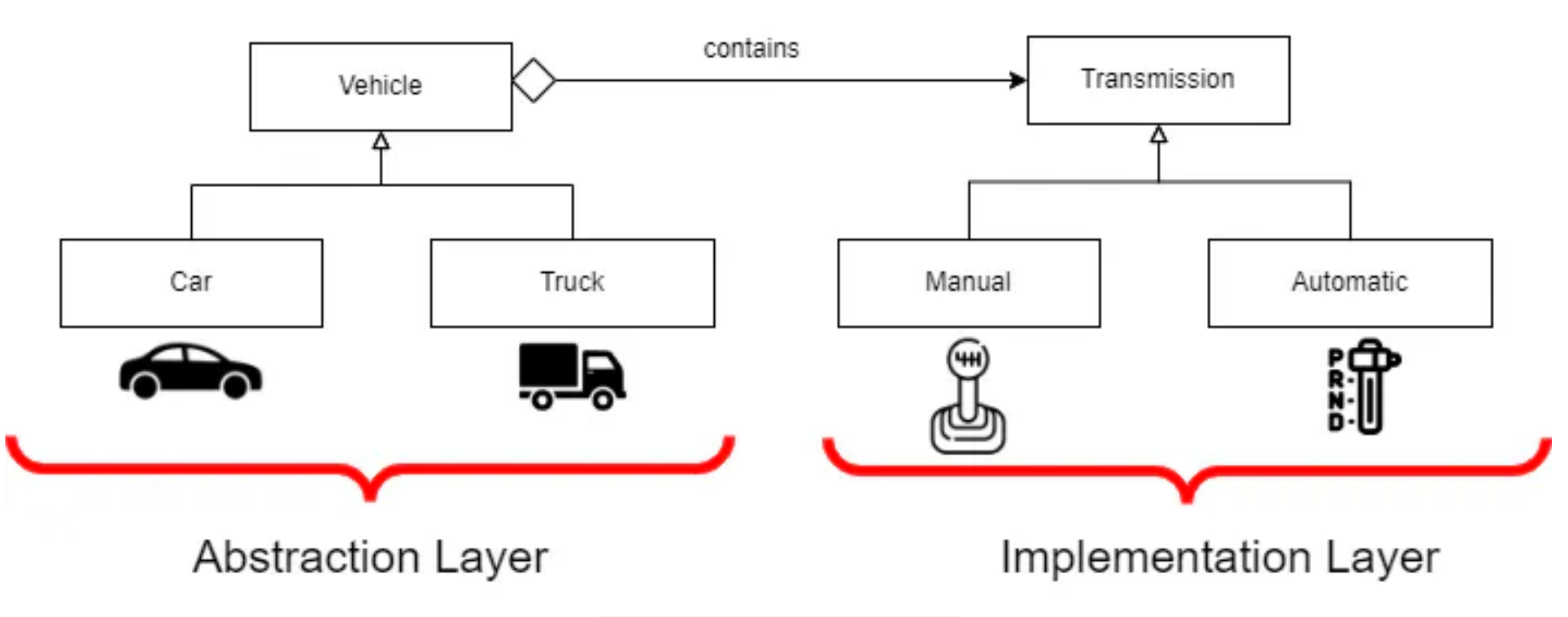
As you try to introduce more types of vehicles like `Bus` or `motorcycle`, or more types of transmission systems like `SemiAutomatic`, the number of classes will grow exponentially. This will make the system unwieldy and hard to maintain.

Bridge Pattern Solution

This problem occurs because we are trying to extend our `Vehicle` class in multiple dimensions like vehicle type and vehicle transmission mode. This issue can be solved using the Bridge Pattern.

Bridge Pattern solves this problem by relating these two dimensions by using class composition instead of class inheritance. The classes will be separated into two different class hierarchies.

- **Abstraction Layer** (**Vehicle Hierarchy**):
  - Base `Vehicle` class with a reference to a `Transmission` object.
  - Subclasses like `Car`, `Truck`, `Bus`, `Motorcycle`, etc., extending `Vehicle`.
- **Implementation Layer** (**Transmission Hierarchy**):
  - `Transmission` interface with methods like `gearUp()`, `gearDown()`.
  - Implementations like `Manual`, `Automatic`, and `SemiAutomatic`.



Bridge Pattern - Solution

This separation allows you to introduce new vehicle types or transmission systems independently without affecting the other hierarchy, reducing the number of classes dramatically.

Abstraction and Implementation

Abstraction

The Bridge Pattern defines abstraction as the high-level layer with which the client interacts. It defines the abstract interface and keeps a reference to an implementation layer object.

In the Vehicle Example:

The class Vehicle is an abstract one. It is a representation of a generic vehicle that has a reference to an implementation layer Transmission object. Subclasses of Vehicle, such as Truck, Bus, and Car, are also included in it. Although they may have additional features or behaviors of their own, they nonetheless inherit high-level behaviors.

Implementation

The concrete implementation of the abstraction's interface is provided by the implementation layer, which is independent of the abstraction. This layer carries out the low-level tasks.

In the Vehicle Example:

The implementation layer consists of the Transmission interface and its implementations (Manual, Automatic, SemiAutomatic). These classes offer particular functions for different transmission systems, like gear shifting.

Real-world Analogy

Bridge Pattern - Real-world analogy

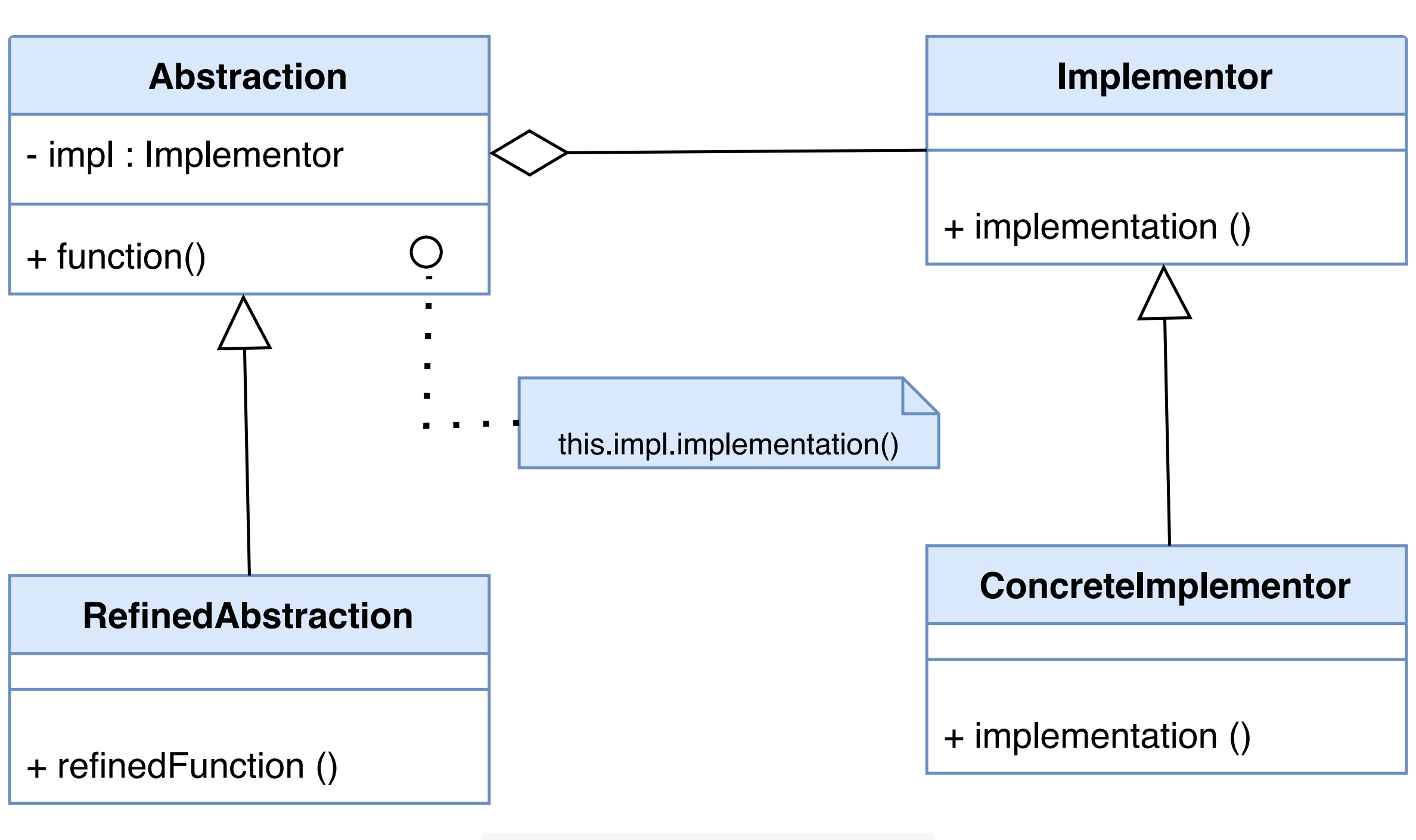
This illustration image shows a simple real-world example of a Bridge Pattern by using a remote control system along with multiple electronic devices.

- In this illustration, the universal remote control system acts as an abstraction layer of the Bridge Pattern. It provides some of the basic functionalities like play, stop, volume up or volume down, etc, but does not directly handle the specific operations of each device.
- The electronic devices acts as an Implementation layer where each device has its own particular way of handling different functionalities,

This configuration exemplifies the fundamental idea of the Bridge Pattern: you may add new devices or modify current ones without changing the design of the remote control. The remote (abstraction) can operate many devices (implementations). Similar to this, the Bridge Pattern in software architecture enables an abstraction to communicate with different implementations, promoting both layers' autonomous evolution and adaptability.

Structure of Bridge Pattern

- **Abstraction**: An abstract class or interface is usually used for this. It has an Implementor reference. It is represented as a class in the UML that has an aggregation relationship to the Implementor.
- **Refined Abstraction**: A concrete class that expands on the Abstraction is called `RefinedAbstraction`. In the UML, it appears to inherit from the Abstraction class.
- **Implementor**: A concrete implementor's methods are defined by an interface. It is represented as an interface in UML.
- **ConcreteImplementor**: This is a concrete class that implements the interface `Implementor`. They have a realization (implementation) relationship with the `Implementor`.



Bridge Pattern - Class Diagram

Implementation of Bridge Pattern

Looking back to our Vehicle class example, we will look at the implementation of that example in multiple programming languages. Are you excited to see the implementation? Let's dive in!

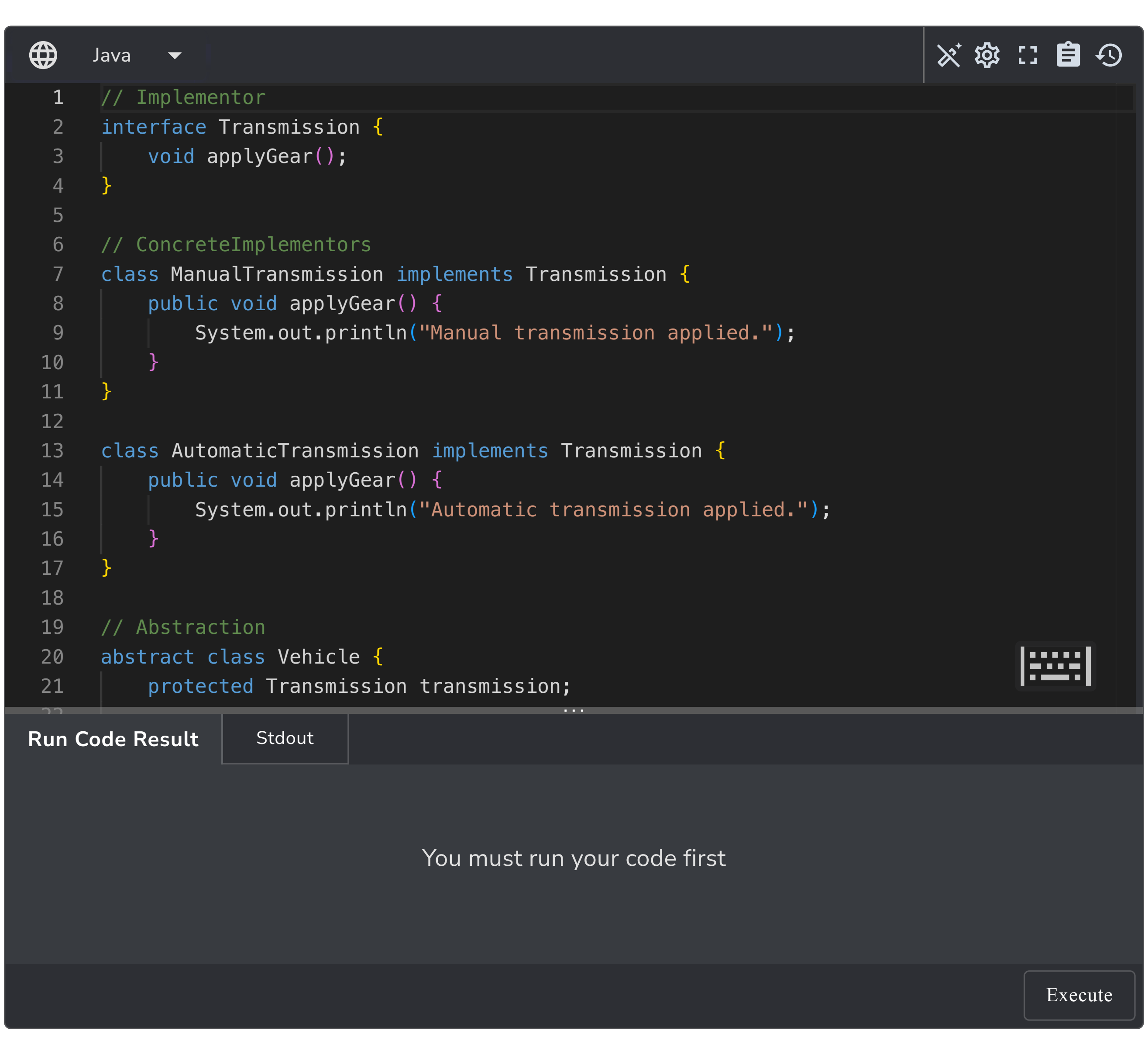
Pseudocode

```
// Implementor
INTERFACE Transmission
    METHOD applyGear()

// ConcreteImplementors
CLASS ManualTransmission IMPLEMENTS Transmission
    METHOD applyGear()
        // Implementation for manual transmission

CLASS AutomaticTransmission IMPLEMENTS Transmission
    METHOD applyGear()
        // Implementation for automatic transmission

// Abstraction
CLASS Vehicle
    PROTECTED field transmission: Transmission

    CONSTRUCTOR Vehicle(transmission)
        this.transmission = transmission

    METHOD applyTransmission()

// RefinedAbstraction
CLASS Car EXTENDS Vehicle
    METHOD applyTransmission()
        transmission.applyGear()

CLASS Truck EXTENDS Vehicle
    METHOD applyTransmission()
        transmission.applyGear()

// Client code
manual = NEW ManualTransmission()
car = NEW Car(manual)
car.applyTransmission()
```

In this pseudocode, we have followed the following steps:

1. **Define Implementor Interface**:
   - `Transmission` interface with a method like `applyGear()`.
2. **Create Concrete Implementors**:
   - Implement the `Transmission` interface in classes like `ManualTransmission` and `AutomaticTransmission`.
3. **Establish Abstraction Class**:
   - `Vehicle` class containing a reference to `Transmission`. It delegates transmission-related operations to the `Transmission` object.
4. **Develop Refined Abstractions**:
   - Subclasses of `Vehicle` like `Car` and `Truck`, which use the `Transmission` interface methods.
5. **Client Interaction**:
   - The client combines a `Vehicle` with a specific `Transmission` type and invokes methods, which are then delegated to the `Transmission` object.

Implementation



```
// Implementor
interface Transmission {
    void applyGear();
}

// ConcreteImplementors
class ManualTransmission implements Transmission {
    public void applyGear() {
        System.out.println("Manual transmission applied.");
    }
}

class AutomaticTransmission implements Transmission {
    public void applyGear() {
        System.out.println("Automatic transmission applied.");
    }
}

// Abstraction
abstract class Vehicle {
    protected Transmission transmission;
```

You must run your code first

Application of Bridge Pattern

We have understood the concept of bridge pattern, but, when to use it? Let's look at some of the scenarios where bridge pattern can help us out.

- **When to Expect Variations Between Implementations and Abstractions**: The Bridge Pattern is a suitable option if you know that the low-level logics (implementation) and the high-level logic (abstraction) are likely to change or extend separately.
- **In Order to Prevent Permanent Binding Between Implementation and Abstraction**: when you wish to prevent a permanent coupling between an abstraction and its implementations and the abstraction can work with many other implementations.
- **To Keep Implementation Details Private from Clients**: It's advantageous to keep certain implementation details secret from clients. Just what is required is exposed by the abstraction layer; the implementation layer is responsible for the remaining information.
- **Refactoring a Monolithic Class Hierarchy**: The Bridge Pattern can assist in dividing a class hierarchy into discrete functional groups if it comprises multiple distinct functionality.
- **Platform independence**: Bridge Pattern is helpful when creating a system that can operate on several different platforms and allows you to expand or change the portions that are platform-specific.

Pros and Cons

Every picture has two sides, good and bad. In the same way Bridge pattern has some advantages and some disadvantages. Here's a table summarizing the pros and cons of the Bridge Pattern:

Pros	Cons
Decouples Abstraction and Implementation: Allows independent modifications to abstractions and implementations.	**Increased Complexity**: Adding more classes and interfaces can complicate the code structure.
Improved Extensibility: It allows easy extension to both abstraction and implementation layers without affecting each other.	**Initial Overhead**: As compared to simpler designs, it is complex to setup initially.
Improved Maintainability: Maintenance of the code is easy since changing implementation layer does not affect the client side code.	**Understanding Overhead**: For the proper and complete implementation, the pattern must be understood properly.
Sharing of Implementation: Redundancy can be used by using the same implementations with multiple abstractions.	**Performance Considerations**: The abstraction layer can add an extra level of indirection.
Platform Independence: It allows creation of the systems that are platform independent.	

This table illustrates how the Bridge Pattern's improved flexibility and maintainability are balanced against the trade-off of increased complexity and upfront design and understanding costs.

← Previous
Adapter Pattern
Next →
Composite Pattern
☐ Mark as Completed